

MyVT-Gacha

Full Code Review Report

Repository	github.com/eri-james/MyVT-Gacha
Review Date	May 16, 2026
Branch	main
Files Reviewed	7 files (~6,860 lines)
Live Demo	eri-james.github.io/MyVT-Gacha

Prepared by **Rashiko Dashboard** for Wiwi

1. Executive Summary

This report presents a comprehensive code review of the **MyVT-Gacha** project, a browser-based idle gacha game featuring 319 Malaysian VTubers. The codebase consists of approximately 6,860 lines of vanilla HTML, CSS, and JavaScript spread across 7 files, with no build tools, frameworks, or external dependencies beyond a single JSON data file. The game is deployed via GitHub Pages and uses localStorage for all client-side persistence.

The review identified **3 critical bugs** that directly impact gameplay mechanics, **6 high-priority issues** affecting correctness and user experience, **6 medium-priority items** related to code quality and maintainability, and **5 low-priority suggestions** for long-term improvement. The three critical bugs are particularly urgent: the tick loop divides income by an extra factor of 60, making real-time earnings 60x slower than offline earnings; the studio leveling function contains a potential infinite loop; and the SSR streak counter is inflated during multi-pull guarantee pulls.

Despite these issues, the overall architecture is sound. The IIFE module pattern provides clean namespacing, the save/load system includes version migration support, and the game features a surprisingly complete set of mechanics for a vanilla JS project: gacha pulls with pity, character leveling and ascension, studio management with station assignment, daily login rewards, milestones, achievements, and activity logging. The code is readable and well-organized, making it straightforward to address the identified issues.

Category	Count	Status
Critical Bugs	3	Fix Immediately
High-Priority Issues	6	Fix Before Next Release
Medium-Priority Issues	6	Schedule for Sprint 6
Low-Priority Suggestions	5	Backlog

Table 1: Issue Summary by Severity

2. Repository Overview

The project follows a flat file structure with no build pipeline, no package manager, and no bundling. All JavaScript modules are loaded via script tags in index.html and communicate through global IIFE return values. This simplicity is appropriate for the current scale and zero-budget constraint, but will become a limitation as the codebase grows.

File	Lines	Purpose
index.html	340	Page structure, 5 tabs, 3 modals
css/style.css	2,089	Full responsive stylesheet

js/data.js	33	Character roster loader (DataLoader)
js/game.js	~490	Core state, tick loop, save/load
js/gacha.js	~170	Pull logic, rates, pity, banners
js/characters.js	~104	Leveling, ascension, shards
js/ui.js	~1,396	All DOM rendering, modals, animations
data/characters.json	2,235	319 VTuber records

Table 2: File Inventory

2.1 Architecture Pattern

All JavaScript modules use the **Revealing Module Pattern** (IIFE with a public API object). This provides clean namespacing and encapsulation without requiring a build step. The dependency graph is linear and unidirectional: DataLoader loads data, Game manages state, Gacha handles pulls, Characters manages progression, and UI orchestrates everything. This is a reasonable architecture for a project of this size and complexity.

However, all modules are **global singletons** loaded via script tags. There is no dependency injection, no import resolution, and no lazy loading. The loading order matters: DataLoader must load before any other module uses it. This creates an implicit coupling that is not enforced by any tooling, meaning a simple reorder of script tags could introduce subtle bugs that only appear in certain browser caching states.

2.2 Key Technical Decisions

- **localStorage persistence** with version migration (currently v2). Save data includes player state, currencies, character collection, pull history, activity log, and achievements.
- **1-second interval tick loop** for real-time resource generation. Each tick calculates income across all assigned stations and adds fractional resources.
- **12-hour offline earnings cap** with per-minute income calculation. The cap prevents excessive rewards from extended absences.
- **3-rate gacha system** (Normal 82%, SR 15%, SSR 3%) with a 50-pull pity threshold guaranteeing SSR. Multi-pull (10x) guarantees at least one SR.
- **Character ascension** from Normal to SR to SSR using star fragments earned from stations or obtained from duplicate pulls.
- **Export/import save codes** via base64 encoding, allowing players to transfer progress between devices manually.

3. Critical Bugs

These bugs directly affect core gameplay mechanics and must be fixed immediately. They impact resource generation, game stability, and statistical accuracy.

3.1 Bug CR-01: Double Division in Tick Loop

IMPACT: Real-time income is 60x slower than displayed offline rates

In **game.js**, the **startTickLoop()** function calculates per-second income by calling **calculatePerMinuteIncome(1/60)**, which correctly returns earnings for one second of game time. However, the function then divides each resource value by 60 again before adding it to the state. This results in the player earning only 1/3600th of the intended per-minute rate per second, or equivalently, 1/60th of the correct per-minute income per minute of real time.

The offline earnings path (**claimOfflineEarnings**) calls **calculatePerMinuteIncome** directly and adds the result without the extra division, so offline earnings are calculated correctly. This creates a 60x discrepancy between real-time and offline income rates. A player who earns 100 stars per minute offline would earn only ~1.67 stars per minute while actively playing.

Location: game.js, startTickLoop(), line inside setInterval callback

Current code:

```
const earnings = calculatePerMinuteIncome(1/60); // already per-second
state.currencies.stars += earnings.stars / 60; // BUG: divides again
```

Fix:

```
const earnings = calculatePerMinuteIncome(1/60); // per-second
state.currencies.stars += earnings.stars; // no extra division
```

Alternatively, call **calculatePerMinuteIncome(1)** and then divide by 60, which is semantically clearer: "calculate one minute of income, then take 1/60th for one second."

3.2 Bug CR-02: Infinite Loop in addStudioExp()

IMPACT: Game freezes when studio gains enough EXP for multiple level-ups

In **game.js**, the **addStudioExp()** function uses a while loop to handle multi-level gains. However, the **nextLevel** variable is computed **before** the while loop and is never reassigned inside it. After the first iteration levels up the studio, the loop condition still checks against the old threshold, which has already been exceeded. The inner checkNext variable is recalculated, but because state.studio.level does not advance (nextLevel.level is the old value), checkNext always matches the same level as nextLevel, creating an infinite loop.

This bug would trigger whenever a player claims offline earnings that grant enough studio EXP to level up more than once. Given that the studio EXP thresholds range from 200 to 60,000 and the offline cap is 12 hours, a player with multiple high-rarity characters assigned could easily accumulate enough EXP to trigger this condition.

Location: game.js, addStudioExp()

Fix: Move the nextLevel lookup inside the while loop:

```
function addStudioExp(amount) {
  state.studio.exp += amount;
  while (true) {
    const next = STUDIO_LEVELS.find(l => l.level === state.studio.level + 1);
    if (!next || state.studio.exp < next.exp) break;
    state.studio.level = next.level;
    if (next.level >= STUDIO_LEVELS.length) break;
  }
}
```

3.3 Bug CR-03: SSR Streak Tracking Error in Multi-Pull

IMPACT: SSR streak statistic is artificially inflated

In **gacha.js**, the **pullMulti()** function includes a guarantee that the 10th pull in a multi-pull will be at least SR. When this guarantee activates, the code unconditionally increments **state.stats.ssrStreak** regardless of whether the forced pull is SSR or SR. If the forced pull ends up as SR (which has an 85% chance), the streak counter increments without the player actually pulling an SSR, leading to an inflated "best SSR streak" stat.

Location: gacha.js, pullMulti(), the i===9 guarantee block

Fix: Only increment ssrStreak when the forced variant is actually SSR:

```
if (i === 9 && !hasSR) {
  variant = Math.random() < 0.85 ? "sr" : "ssr";
  if (variant === "ssr") {
    state.pity.count = 0;
    state.stats.ssrStreak++;
    if (state.stats.ssrStreak > state.stats.bestSsrStreak)
      state.stats.bestSsrStreak = state.stats.ssrStreak;
  } else {
    state.stats.ssrStreak = 0; // SR resets streak
  }
}
```

4. High-Priority Issues

These issues do not cause crashes but lead to incorrect behavior, inconsistent UX, or potential data integrity problems. They should be addressed before the next feature sprint.

4.1 HP-01: Featured Banner Characters Regenerate on Every Display

In **ui.js**, the **updateBannerInfo()** function calls **Gacha.generateFeatured()** every time it runs, creating a new set of random featured characters. This function is called on every tab switch and after every pull. Meanwhile, the actual pull logic in **rollCharacter()** uses the **featuredCharacters** array set by **setupFeaturedBanner()** during init. The result is that the banner description shows different featured characters than what the player is actually pulling, causing confusion and undermining trust in the gacha system.

Fix: Store the featured character names during setup and reuse them for display, or call **generateFeatured()** only once per session/banner switch.

4.2 HP-02: **pullSingle()** Does Not Trigger State Change Callback

In **gacha.js**, the **pullSingle()** function includes the comment `"// Will trigger via save"` next to an empty **if (Game.onStateChange) {}** block. This is a no-op. While the tick loop will eventually update the UI, there is a visible delay of up to 1 second after a single pull before the UI reflects the new state. Multi-pull calls **Game.save()** which triggers auto-save but does not call the state change callback either. The UI only updates because of the subsequent **updateUI()** call in **handlePull()** within **ui.js**.

4.3 HP-03: Potential XSS via onclick Attribute Injection

In **ui.js**, the **showCharacterDetail()** function builds HTML strings with inline onclick handlers using string interpolation: **onclick="UI.levelUpChar('\${slug}')"**. If a character slug were to contain single quotes or other HTML special characters, this would create a DOM-based XSS vulnerability. While the current character data appears to use safe slugs (lowercase alphanumeric with hyphens), this is a fragile assumption. Future data updates or a character name with special characters could introduce a vulnerability.

Fix: Use **addEventListener()** instead of inline onclick, or sanitize slugs before interpolation. The pull card creation in **createPullCard()** already uses **addEventListener** correctly and should be used as the pattern reference.

4.4 HP-04: DataLoader Race Condition

The **DataLoader.load()** function is async but has no guard against concurrent calls. If multiple modules call **DataLoader.get()** before the fetch completes, they receive an empty array. Currently this is mitigated by the sequential init flow in **ui.js** (**await DataLoader.load()** before anything else), but any future refactoring that adds parallel initialization could trigger this race condition. Additionally, the fetch response is not checked for a successful status code; a 404 or 500 error would throw an unhandled exception when calling **.json()**.

4.5 HP-05: Deprecated escape/unescape in Save Import

In `game.js`, the `importSaveCode()` function uses the deprecated `escape()` and `unescape()` functions for UTF-8 base64 encoding/decoding. While these still work in current browsers, they are officially deprecated in the ECMAScript specification and may be removed in future engine versions. The modern replacement is `TextDecoder` for decoding UTF-8 byte arrays.

Fix: Replace with: `const bytes = Uint8Array.from(atob(code), c => c.charCodeAt(0)); const json = new TextDecoder().decode(bytes);`

4.6 HP-06: Offline Earnings Discrepancy Creates Trust Issues

Because of Bug CR-01, the offline earnings calculator and the real-time tick loop produce wildly different results for the same station setup. The offline claim button might show "+500 Stars for 60 minutes offline" but the player observes only ~8 stars per minute while actively watching. This discrepancy, even after CR-01 is fixed, reveals that the offline calculation path and the tick loop path were developed independently and should be unified into a single income calculation function used by both code paths.

5. Medium-Priority Issues

These items affect code maintainability, performance, and developer experience. They should be addressed in a dedicated refactoring sprint but are not urgent.

5.1 MP-01: Duplicated Income Calculation Logic

The per-station income formula (`baseRate * variantMult * levelBonus * stationMult`) is duplicated in two places: `calculatePerMinuteIncome()` in `game.js` and `getStationIncome()` in `game.js`. These two functions compute the same value but serve different callers. Any future change to the income formula (e.g., adding a new multiplier) would need to be updated in both places. This violates the DRY principle and creates a maintenance burden.

Fix: Have `calculatePerMinuteIncome` call `getStationIncome` internally.

5.2 MP-02: No Input Validation on Save Import

The `importSaveCode()` function validates only that the decoded JSON has `version` and `playerId` fields. It does not validate the structure of nested objects like currencies, characters, or studio. A malformed or maliciously crafted save code could set negative currency values, corrupt the characters map, or inject unexpected properties. For a game with competitive elements like achievement tracking and collection stats, this is a data integrity risk.

5.3 MP-03: Full DOM Re-render on Every Tick

The **updateUI()** function in `ui.js` is called every second by the tick loop via the `onStateChange` callback. This function updates every resource display, every stat counter, the daily login button state, the pity counter, all pull button disabled states, and conditionally re-renders the studio tab, collection progress, and profile tab. This is approximately 15-20 DOM writes per second. For a game running primarily in the background tab, this is wasteful. A simple optimization would be to only update the resource counters on each tick and defer the full UI refresh to user-initiated events (tab switches, pulls).

5.4 MP-04: Magic Numbers Throughout Codebase

Numerous constants are hardcoded as magic numbers rather than being defined as named constants at the top of their respective modules. Examples include the pull costs (100, 1000), pity threshold (50), collection page size (40), pull log cap (50), activity log cap (50), auto-save interval (30000ms), and offline cap (12 hours). While some of these are defined as module-level constants, others are used inline. This makes tuning game balance more difficult than necessary.

5.5 MP-05: No Debounce on Auto-Save During Rapid Actions

The auto-save interval is 30 seconds, but several actions (level-up, ascend, station assignment) call **Game.save()** immediately. During rapid actions like repeated level-ups, this means multiple synchronous `localStorage` writes within milliseconds. While `localStorage` writes are fast on modern browsers, they block the main thread and could cause micro-stutters on low-end devices. A debounced save that coalesces rapid writes would be more efficient.

5.6 MP-06: Characters Module Does Not Trigger UI Refresh

In `characters.js`, the **levelUp()** and **ascend()** functions call **Game.save()** but do not call the state change callback. The UI only refreshes when the calling code in `ui.js` explicitly calls **updateUI()**. This works currently because all callers do so, but it is an implicit contract that could break if new callers are added. The pattern should be consistent: either always trigger the callback from within the action function, or never trigger it and always require the caller to do so.

6. Low-Priority Suggestions

These items are quality-of-life improvements that would benefit the project long-term but are not necessary for the current playable state.

6.1 LP-01: No Build Tooling or Minification

The project ships ~6,860 lines of uncompressed HTML, CSS, and JavaScript. With a simple build step using `esbuild` (which requires no configuration for vanilla JS), the JavaScript could be bundled and minified to roughly 30-40% of its current size. This would improve initial load time, especially on mobile connections. A single build step would also enable ES module imports, eliminating the global

singleton pattern and the implicit load-order dependency.

6.2 LP-02: CSS File Size at 2,089 Lines

The stylesheet is a single monolithic file covering all components, responsive breakpoints, and animation keyframes. As the game adds more features, this file will become increasingly difficult to maintain. Consider splitting into component-specific files (e.g., `_cards.css`, `_studio.css`, `_modals.css`, `_responsive.css`) and using CSS custom properties for theming. This would also enable easier dark mode implementation in the future.

6.3 LP-03: Touch Event Handling for Mobile

Several interactive elements use only click event listeners. On mobile devices, this works but introduces a 300ms delay before the event fires (on older browsers) and lacks touch-specific feedback like haptic feedback or press states. The pull animation overlay could benefit from touch-action CSS and pointer events for smoother mobile interaction.

6.4 LP-04: Potential Memory Leak in Toast System

Toast notifications are created, appended to the DOM, and removed after a timeout. However, if multiple toasts are created rapidly (e.g., during a multi-pull with milestone + achievement notifications), the container can accumulate elements before their removal timers fire. Each toast is a full DOM element with text content. A maximum concurrent toast limit (e.g., 3) with immediate removal of the oldest would prevent unbounded DOM growth.

6.5 LP-05: No Automated Tests

The project has no test suite. Given the game mechanics involve probability (gacha rates, pity), resource calculations (income formulas, level costs), and state transitions (ascension, migration), even a small set of unit tests for the core calculation functions would prevent regressions. A testing framework like Vitest would integrate well with a minimal build step and could run in GitHub Actions as part of the CI pipeline.

7. Full Issue Tracker

ID	Severity	File	Title
CR-01	Critical	game.js	Double division in tick loop (60x slow)
CR-02	Critical	game.js	Infinite loop in addStudioExp()

CR-03	Critical	gacha.js	SSR streak inflated on multi-pull
HP-01	High	ui.js	Featured banner regenerates on display
HP-02	High	gacha.js	pullSingle() no-op state change
HP-03	High	ui.js	XSS via onclick slug injection
HP-04	High	data.js	DataLoader race condition
HP-05	High	game.js	Deprecated escape/unescape
HP-06	High	game.js	Offline vs real-time discrepancy
MP-01	Medium	game.js	Duplicated income calculation
MP-02	Medium	game.js	No save import validation
MP-03	Medium	ui.js	Full DOM re-render every tick
MP-04	Medium	All	Magic numbers not centralized
MP-05	Medium	game.js	No debounce on rapid saves
MP-06	Medium	characters.js	No UI refresh after actions
LP-01	Low	All	No build tooling or minification
LP-02	Low	style.css	Monolithic 2089-line CSS file
LP-03	Low	ui.js	No touch event optimization
LP-04	Low	ui.js	Toast container unbounded growth
LP-05	Low	All	No automated test suite

Table 3: Complete Issue Tracker (20 items)

8. Recommendations

8.1 Immediate Actions (This Week)

- **Fix CR-01:** Remove the extra /60 division in startTickLoop(). This is a one-line fix with massive gameplay impact.
- **Fix CR-02:** Rewrite addStudioExp() with the nextLevel lookup inside the while loop.
- **Fix CR-03:** Move ssrStreak++ inside an if-SSR block in the multi-pull guarantee.
- **Fix HP-01:** Store featured characters once and reuse for both display and pull logic.
- **Fix HP-03:** Replace inline onclick handlers with addEventListener in showCharacterDetail().

8.2 Sprint 6 Priorities

- Unify the income calculation into a single function used by both the tick loop and offline earnings paths.
- Add status code checking to DataLoader.fetch() and error handling for failed loads.
- Replace deprecated escape/unescape with TextDecoder in importSaveCode().
- Add basic input validation to the save import function (check for negative values, required fields).
- Optimize updateUI() to only update resource counters on each tick, deferring heavy re-renders.

8.3 Long-Term Improvements

- Introduce a minimal build step (esbuild) for bundling, minification, and ES module support.
- Split the CSS into component-specific files and add CSS custom properties for theming.
- Add a small test suite covering core calculation functions (income formulas, gacha rates, level costs).
- Implement a proper cloud sync solution using a free backend (Supabase, Firebase) to replace manual save codes.
- Consider adding a service worker for offline support and faster subsequent page loads.

This review was prepared by the Rashiko Dashboard AI assistant based on a static analysis of the main branch source code at github.com/eri-james/MyVT-Gacha. No runtime testing or dynamic analysis was performed. All line references are approximate and may shift with future commits.